

# Algorithmen und Datenstrukturen

Übung 8: Sortieren II

Robert

2026-06-26

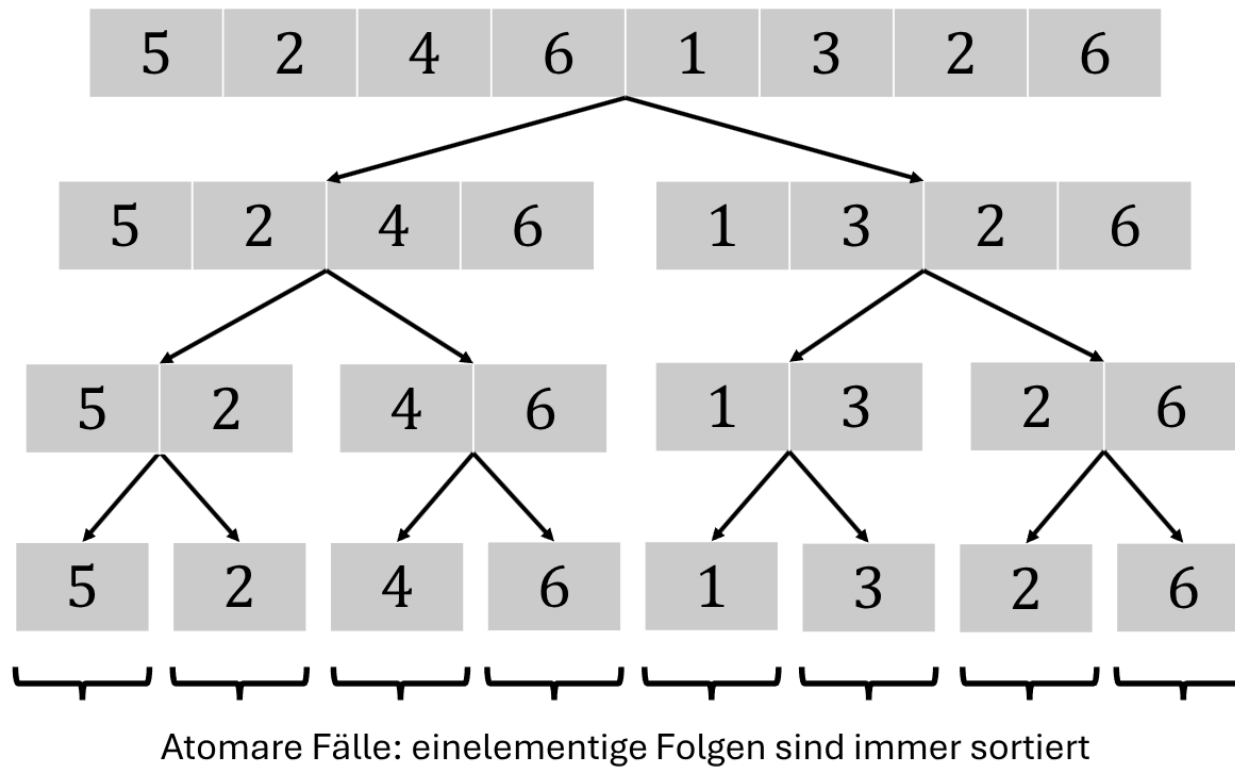
# Wiederholung MergeSort

# MergeSort

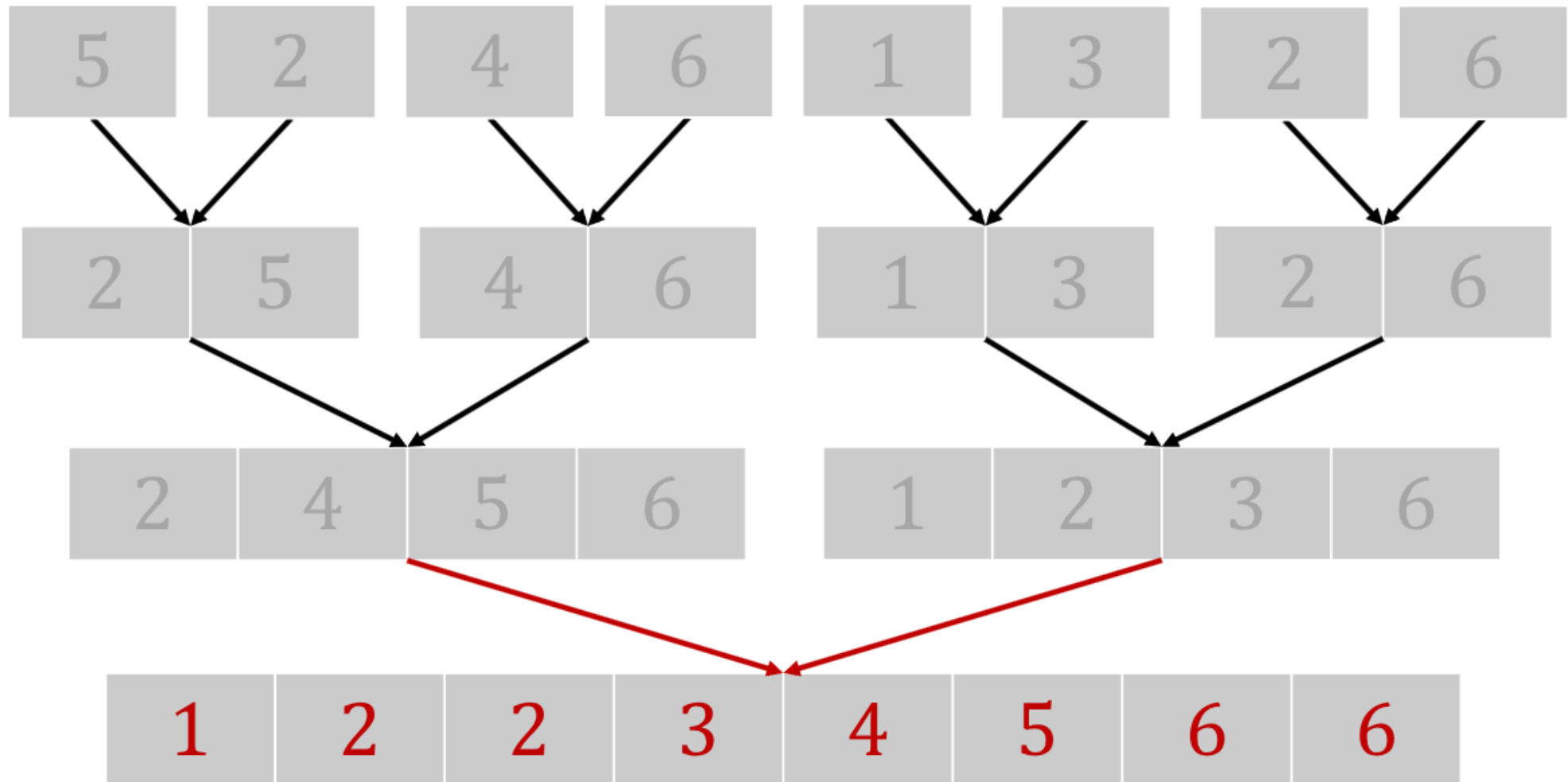
- die Idee von MergeSort ist “Divide and Conquer”
  - “Divide”: Ein großes Problem in mehrere kleine Probleme aufteilen
  - “Conquer”: Die Teilprobleme einzeln und unabhängig voneinander lösen
  - “Merge”: Die Teilprobleme zur Gesamtlösung zusammenführen.

# Schritte

1. ein Array wird solange in der Hälfte aufgeteilt (“divide”), bis am Ende einzelne Elemente übrig bleiben
  - die einzelnen Elemente sind dann automatisch sortiert (“conquer”)



2. führe die sortierten Teilfolgen geordnet zusammen



# Implementation

- der erste Algorithmus führt den ersten Schritt durch:

```
1 def mergeSort(array):
2     if len(array) > 1:
3         mid = len(array) // 2
4         lefthalf = array[:mid] # '[:mid]' = von Anfang bis Mitte
5         righthalf = array[mid:] # '[mid:]' = ab der Mitte bis zum Ende
6                                 # Slicing-Syntax in Python
7         mergeSort(lefthalf)
8         mergeSort(righthalf)
9         merge(array, lefthalf, righthalf)
```

- der zweite Algorithmus `merge` nimmt sich die zwei Hälften (`left`, `right`) und führt sie in das übergebene Array `array` ein:

```
1 def merge(array, left, right):
2     i,j,k = 0,0,0
3     while i < len(left) and j < len(right):
4         if left[i] <= right[j]:
5             array[k] = left[i]
6             i+=1
7         else:
8             array[k] = right[j]
9             j+=1
10        k+=1
11
12    while i < len(left): # falls linkes Teilarray Elemente übrig hat
13        array[k] = left[i]
14        i+=1
15        k+=1
16
17    while j < len(right): # falls rechtes Teilarray Elemente übrig hat
18        array[k] = right[j]
19        j+=1
20        k+=1
```

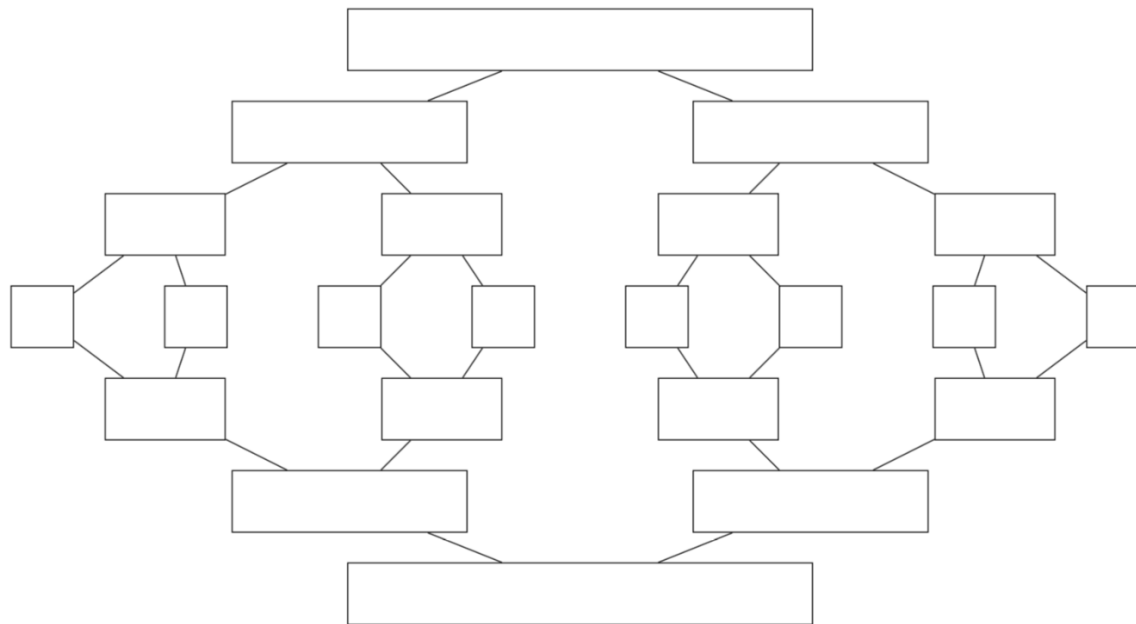
# Aufgabe 1



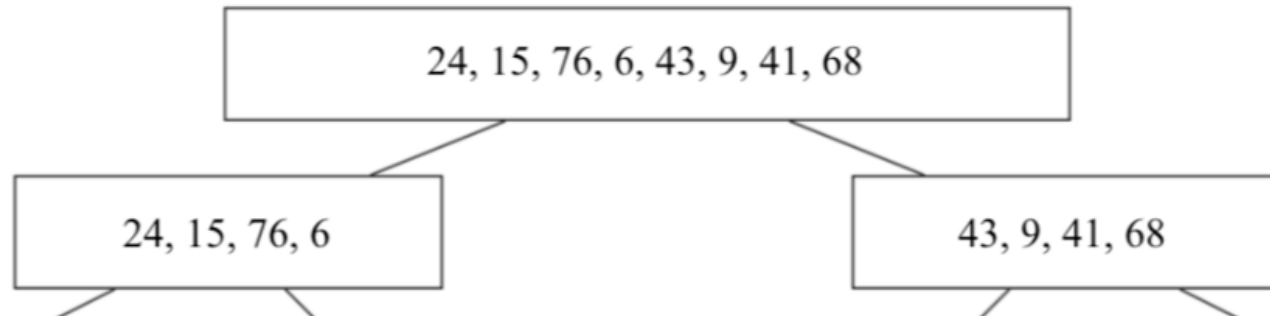
# 1 a)

Sortiere die Zahlenfolge mit MergeSort und trage alle Zwischenschritte in die Vorlage ein.

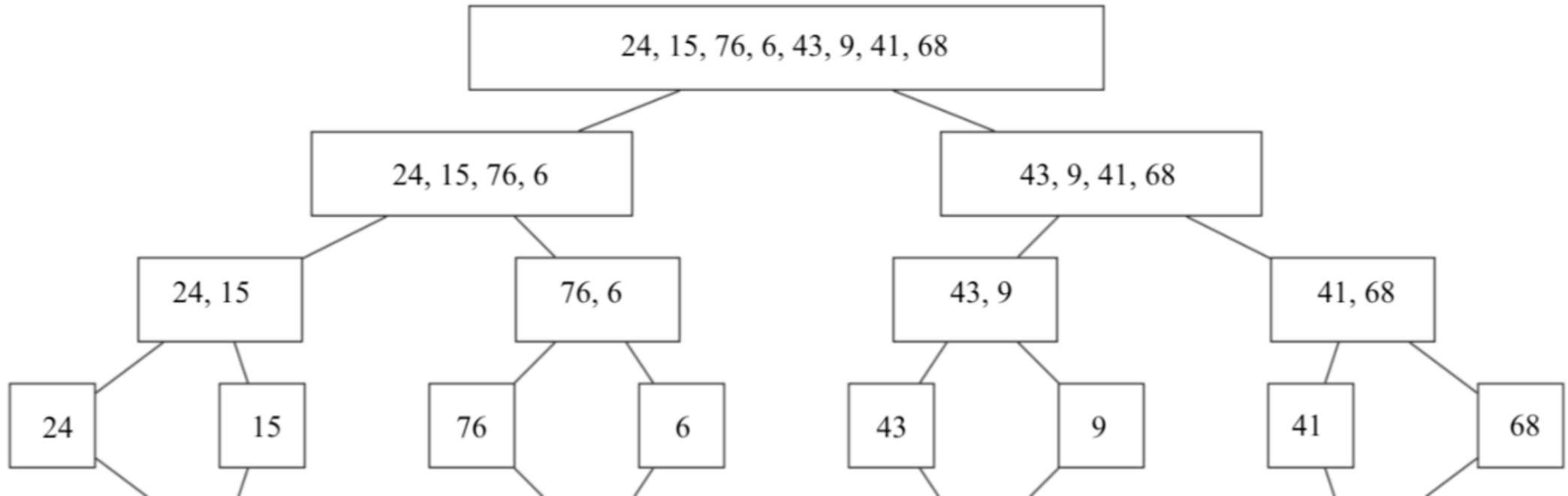
24, 15, 76, 6, 43, 9, 41, 68



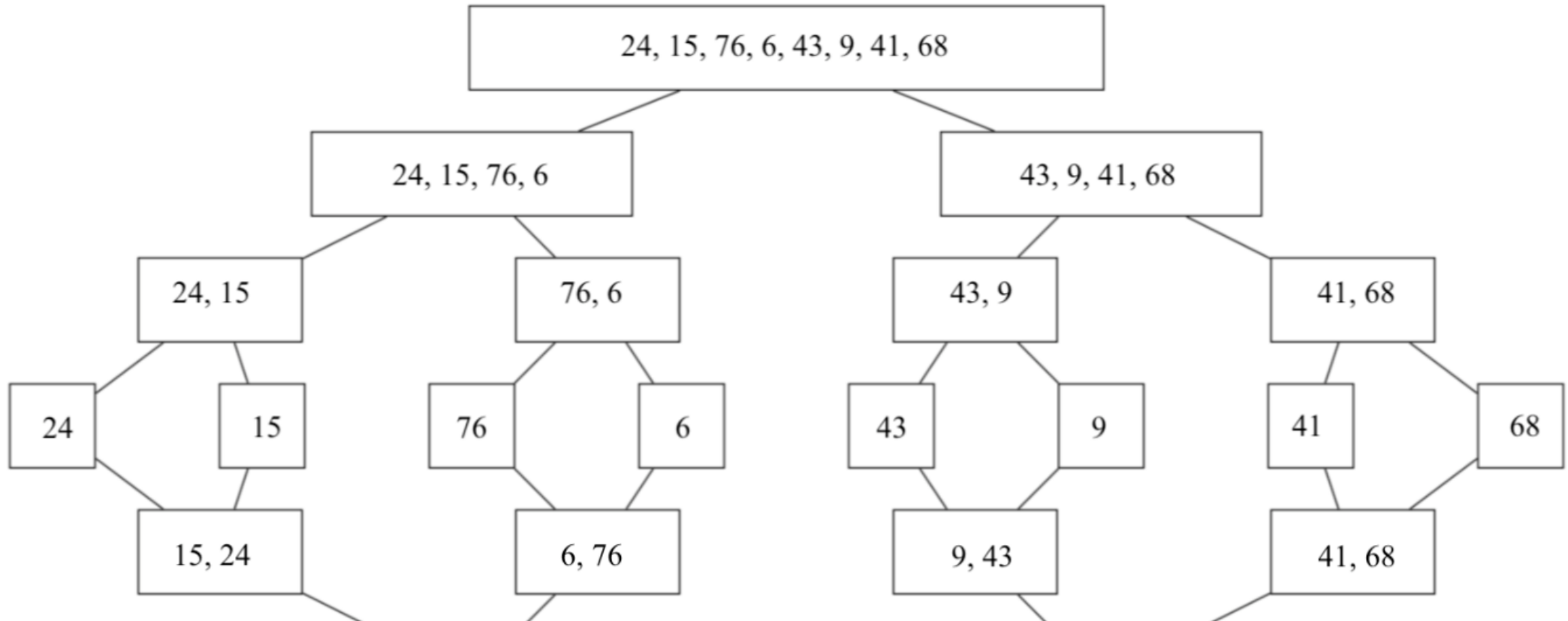
24, 15, 76, 6, 43, 9, 41, 68



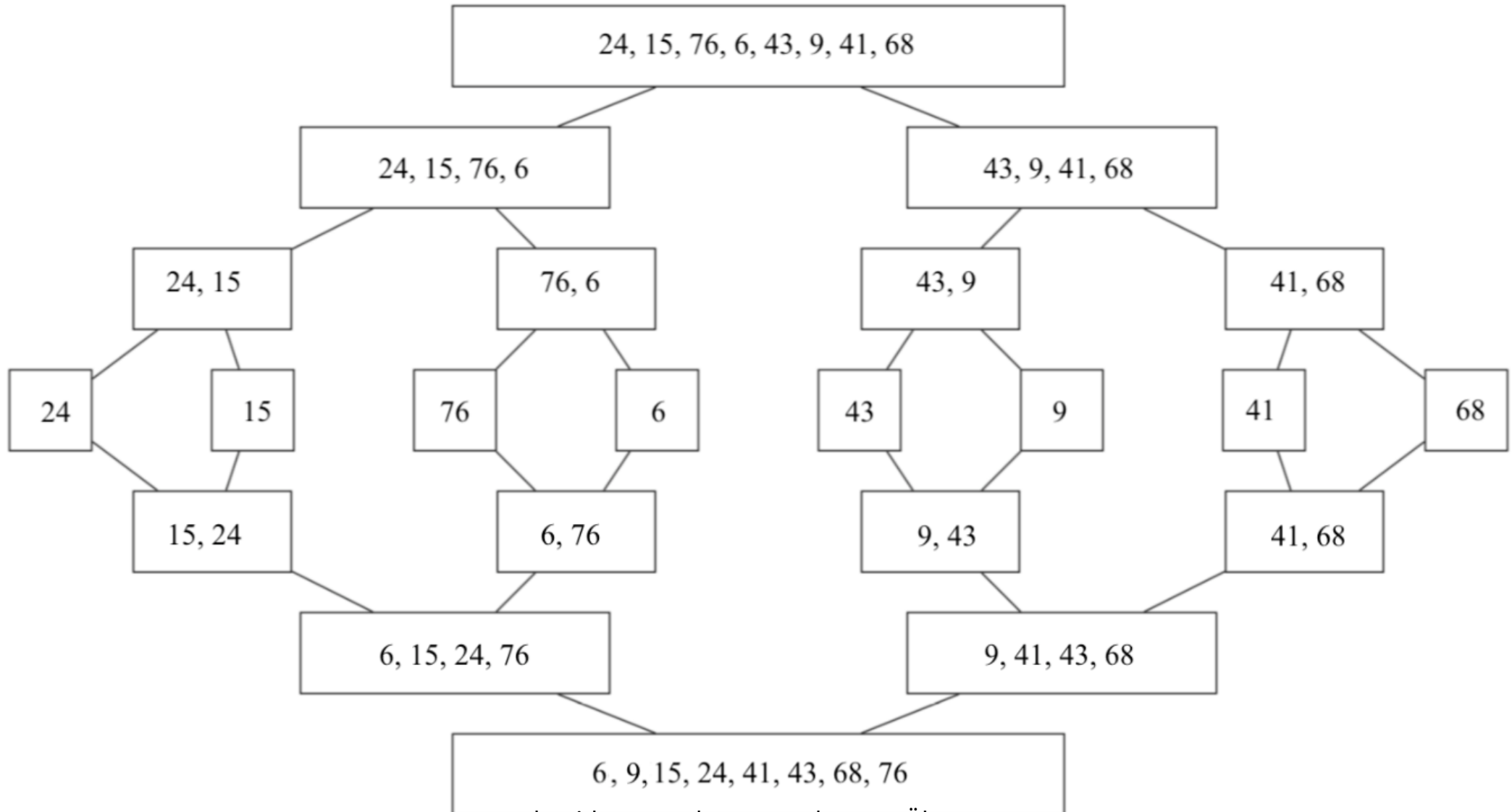
24, 15, 76, 6, 43, 9, 41, 68



24, 15, 76, 6, 43, 9, 41, 68



24, 15, 76, 6, 43, 9, 41, 68



# 1 b)

Wie viele Vertauschungen wurden vorgenommen?

- 0 Vertauschungen
  - MergeSort vertauscht keine Elemente, sondern fügt sie in der richtigen Reihenfolge in das übergebene Array ein

```
1 merge(array, left, right):  
2     ...  
3     if left[i] <= right[j]:  
4         array[k] = left[i]  
5         i+=1  
6     else:  
7         array[k] = right[j]  
8         j+=1  
9     ...
```

# Wiederholung QuickSort

# QuickSort

- QuickSort ist auch ein “Divide and Conquere”-Algorithmus
  1. ein Element aus der Folge wird zum Pivot-Elemente  $p$
  2. die Folge wird in zwei Teilfolgen zerlegt (“divide”)
    - Elemente, die kleiner sind als  $p$ , kommen in die linke Teilfolge
    - Elemente, die größer sind als  $p$ , kommen in die rechte Teilfolge
    - Schritt 2 wird solange rekursiv auf die Teilfolgen aufgerufen, bis in den Teilfolgen nur noch ein Element übrig bleibt
      - Teilfolgen mit einem Element sind bereits sortiert (“conquer”)
  3. die sortierten Teilfolgen werden zusammengefügt (“merge”)



# Implementation

- der erste Algorithmus schaut, ob in der Teilfolge/Array mehr als ein Element ist, ruft die `partition`-Funktion auf und ruft sich selber wieder auf die neuen Teilfolgen auf:

```
1 def quickSort(array, index_begin, index_end):
2     # if-Bedingung stoppt rekursive Aufrufe
3     if index_begin < index_end:
4         # Aufteilung in zwei Teilfolgen mit p als Trennposition
5         p = partition(array, index_begin, index_end)
6
7         # rekursive Aufrufe auf Teilfolgen
8         quickSort(array, begin, p-1)
9         quickSort(array, p+1, end)
```

- der zweite Algorithmus teilt das übergeben Array in zwei Teilfolgen auf:

```
1 def partition(array, index_begin, index_end):
2     pivot = array[index_end] # Pivot-Element ist das letzte Element
3     p = index_begin # Trennposition ist am Anfang
4
5     for i in range(index_begin, index_end):
6         # falls aktuelles Element kleiner als Pivot-Element ist,
7         if array[i] < pivot:
8             # dann wird aktuelles Element mit Element bei
9             # Trennposition getauscht (Menge der kleineren
10            # Elemente wächst)
11            array[p], array[i] = array[i], array[p]
12            # Trennposition wird verschoben
13            p += 1
14
15    # nachdem durch alle Elemente iteriert wurde, wird
16    # Pivot-Element mit Element bei Trennposition getauscht
17    array[p], array[index_end] = array[index_end], array[p]
18    # Index der Trennposition wird zurückgegeben (links
19    # und rechts neue Teilfolgen)
20    return p
```

# Aufgabe 2

## 2 a)

Wie lange muss gesucht werden, um ein bestimmtes Element in einem unsortierten Array zu finden?

- im Worst Case muss jedes Element betrachtet werden, daher hat die Suchzeit die Größe  $n$

## 2 b)

Sortiere das Array mit Quicksort. Das Pivot-Element ist immer das letzte Element.

[37, 16, 9, 42, 51, 17, 71, 56, 89, 39, 6, 3]

- Schritte:
  1. Pivot-Element
  2. Zerlegung in zwei Teilfolgen (divide)
    - rekursiv, bis nur noch einzelne Elemente übrigbleiben (conquer)
  3. Zusammenfügen (merge)

37 16 9 42 51 17 71 56 89 39 6 ③

3 16 9 42 51 17 71 56 89 39 6 ③7

3 16 9 17 ⑥ 37 71 56 89 39 51 42

3 6 9 17 ①6 37 71 56 89 39 51 42

3 6 9 16 17 37 71 56 89 39 51 ④2

3 6 9 16 17 37 39 42 89 71 51 ⑤6

3 6 9 16 17 37 39 42 51 56 89 ⑦1

3 6 9 16 17 37 39 42 51 56 71 89

Wie wird die 42 einsortiert?

|   |   |   |    |    |    |    |    |    |    |    |    |
|---|---|---|----|----|----|----|----|----|----|----|----|
| 3 | 6 | 9 | 16 | 17 | 37 | 71 | 56 | 89 | 39 | 51 | 42 |
|---|---|---|----|----|----|----|----|----|----|----|----|

vorher

|   |   |   |    |    |    |    |    |    |    |    |    |
|---|---|---|----|----|----|----|----|----|----|----|----|
| 3 | 6 | 9 | 16 | 17 | 37 | 39 | 42 | 89 | 71 | 51 | 56 |
|---|---|---|----|----|----|----|----|----|----|----|----|

nachher

Wie kommt es zu dieser Teilfolge?  $\rightarrow [\dots, 89, 71, 51, 56]$

```
1 def quickSort(array, index_begin, index_end):
2     if index_begin < index_end:
3         p = partition(array, index_begin, index_end)
4
5         quickSort(array, begin, p-1)
6         quickSort(array, p+1, end)
```

- die linke Teilfolge von 37 ist bereits sortiert, deshalb wird Quicksort nun auf die rechte Teilfolge angewendet

|   |   |   |    |    |    |
|---|---|---|----|----|----|
| 3 | 6 | 9 | 16 | 17 | 37 |
|---|---|---|----|----|----|

 71 56 89 39 51 ④2

- `array` ist das Array
- `p+1 = 6`, weil die 37 das Pivot-Element war
- `end = 11`, also der Index der letzten Zahl



```
1 def quickSort(array, index_begin, index_end):
2     if index_begin < index_end:
3         p = partition(array, index_begin, index_end)
4
5         quickSort(array, begin, p-1)
6         quickSort(array, p+1, end)
```

- `index_begin = 6` ist kleiner als `index_end = 11`
  - das heißt in der Teilfolge befindet sich mehr als 1 Element, weshalb Quicksort angewendet werden kann
    - die Funktion `partition` wird als nächstes aufgerufen

```

1 def partition(array, index_begin, index_end):
2     pivot = array[index_end]
3     p = index_begin
4
5     for i in range(index_begin, index_end):
6         if array[i] < pivot:
7             array[p], array[i] = array[i], array[p]
8             p += 1
9
10    array[p], array[index_end] = array[index_end], array[p]
11    return p

```

- das Pivot-Element (42) und der Index **p**, wo das Pivot-Element hin soll (`index_begin = 6`), werden initialisiert



```

1 def partition(array, index_begin, index_end):
2     pivot = array[index_end]
3     p = index_begin
4
5     for i in range(index_begin, index_end):
6         if array[i] < pivot:
7             array[p], array[i] = array[i], array[p]
8             p += 1
9
10    array[p], array[index_end] = array[index_end], array[p]
11    return p

```



- sowohl die 71, 56 und die 89 sind alle größer als die 42
  - die Bedingung ist nicht erfüllt und es wird weiter iteriert bis zur 39

```

1 def partition(array, index_begin, index_end):
2     pivot = array[index_end]
3     p = index_begin
4
5     for i in range(index_begin, index_end):
6         if array[i] < pivot:
7             array[p], array[i] = array[i], array[p]
8             p += 1
9
10    array[p], array[index_end] = array[index_end], array[p]
11    return p

```

|   |   |   |    |    |    |    |    |    |    |    |    |
|---|---|---|----|----|----|----|----|----|----|----|----|
| 3 | 6 | 9 | 16 | 17 | 37 | 71 | 56 | 89 | 39 | 51 | 42 |
|---|---|---|----|----|----|----|----|----|----|----|----|

- die 39 ist kleiner als die 42 (Zeile 10), daher muss die 39 in die (zukünftige) linke Hälfte, indem sie mit dem Element an der Stelle  $p = 6$  vertauscht wird (Zeile 11)
- da jetzt die linke Teilfolge um ein Element gewachsen ist, muss auch der Index  $p$ , wo das Pivot-Element eingefügt werden soll, um 1 verschoben werden (Zeile 12)

|   |   |   |    |    |    |    |    |    |    |    |    |
|---|---|---|----|----|----|----|----|----|----|----|----|
| 3 | 6 | 9 | 16 | 17 | 37 | 39 | 56 | 89 | 71 | 51 | 42 |
|---|---|---|----|----|----|----|----|----|----|----|----|

```

1 def partition(array, index_begin, index_end):
2     pivot = array[index_end]
3     p = index_begin
4
5     for i in range(index_begin, index_end):
6         if array[i] < pivot:
7             array[p], array[i] = array[i], array[p]
8             p += 1
9
10    array[p], array[index_end] = array[index_end], array[p]
11    return p

```



- die 51 ist größer als die 42, wodurch die Bedingung nicht erfüllt ist

```

1 def partition(array, index_begin, index_end):
2     pivot = array[index_end]
3     p = index_begin
4
5     for i in range(index_begin, index_end):
6         if array[i] < pivot:
7             array[p], array[i] = array[i], array[p]
8             p += 1
9
10    array[p], array[index_end] = array[index_end], array[p]
11    return p

```

- die Teilfolge wurde durch iteriert und das Pivot-Element muss nur noch mit dem Element an der Stelle  $p = 7$  getauscht werden



- anschließend wird der Index des Pivot-Elements zurückgegeben
- so entsteht also die rechte Teilfolge

## 2 c)

Wie lange dauert es jetzt, wo das Array sortiert ist, ein bestimmtes Element zu finden?

- jetzt kann bei der Suche im geordneten Array wie bei einem binären Suchbaum vorgegangen werden
- das Element in der Mitte wird mit dem gesuchte verglichen und je nachdem, ob es größer oder kleiner ist, wird in der rechten oder linken Hälfte weitergesucht
  - das heißt die Suche braucht nur noch  $\log(n)$  Schritte

## 2 d)

Wann lohnt es sich laufzeittechnisch, ein Array zu sortieren, auf das eine Anwendung zugreift?

- es lohnt sich, ein Array zu sortieren, sobald mehrmals auf das Array zugegriffen wird
- ausschlaggebend ist die Differenz zwischen der Laufzeit des Sortierens und der ersparten Zeit beim Zugriff auf ein sortiertes Array



## 2 e)

Ist QuickSort stabil?

(stabil = gleiche Elemente werden nicht vertauscht)

- nein, QuickSort ist nicht stabil
- Beispiel: das Pivot-Element ist immer das in der Mitte
  - das Array  $[5, 5, 5]$  ist ein Gegenbeispiel, weil die linke und rechte 5 abhängig vom Vergleich entweder beide links oder rechts von der mittleren 5 eingeordnet werden
- bei einem zufälligen Pivot-Element ist QuickSort sehr instabil, weil so irgendein Element in die Mitte gesetzt wird

## 2 f)

Was ist das Worst Case-Szenario von QuickSort mit einer Laufzeit von  $O(n^2)$ ?

- das Worst Case-Szenario tritt dann auf, wenn das gewählte Pivot-Element immer das größte oder kleinste Element der Teilfolge ist
  - das sorgt dafür, dass jeder rekursive Aufruf die Problemgröße immer nur um 1 verringert

- Beispiel:
  - Array  $A = [1, 2, 3, 4, 5, 6, 7]$
  - das letzte Element ist das Pivot-Element
  - Partitionierung:
    1. Schritt: Pivot = 7 → Linker Teil:  $[1, 2, 3, 4, 5, 6]$ , Rechter Teil:  $[]$
    2. Schritt: Pivot = 6 → Linker Teil:  $[1, 2, 3, 4, 5]$ , Rechter Teil:  $[]$
    - ...
    6. Schritt: Pivot = 2 → Linker Teil:  $[1]$ , Rechter Teil:  $[]$
  - Rekursionstiefe: 6 (für  $n=7$ )
  - da das größte Element immer das letzte Element der Teilfolge ist, ist die Rekursionstiefe sehr hoch
  - bei jeder Rekursion nimmt die Anzahl der Vergleiche nur um 1 ab, daher gibt es  $6+5+4+3+2+1=21$  Vergleiche  $\Rightarrow \frac{n(n-1)}{2} \in O(n^2)$

## 2 g)

Was ist die optimale Laufzeit des Algorithmus? Und wie sieht ein Array aus, bei dem diese Laufzeit auftritt?

- die optimale Laufzeit liegt bei  $O(n \cdot \log(n))$ , weil QuickSort das Array immer in zwei gleich große Hälften aufteilt
- Beispiel:
  - Array  $B = [4, 2, 6, 1, 3, 5, 7]$
  - der Median ist das Pivot-Element

- Partitionierung:
  1. Schritt: Pivot = 4 → Linker Teil: [2, 1, 3], Rechter Teil: [6, 5, 7]
  2. Schritt: Pivot = 2 → Linker Teil: [1], Rechter Teil: [3]
  3. Schritt: Pivot = 6 → Linker Teil: [5], Rechter Teil: [7]
- Rekursionstiefe: 3 (für n=7)
- Vergleiche:  $6 + (2 \cdot 2) = 10$  Vergleiche
- Mastertheorem:  $2 \cdot T\left(\frac{n-1}{2}\right) + (n - 1)$  wenn  $n > 1$ 
  - das heist:  $O(n \cdot \log(n))$

## 2 h)

Was ist generell eine gute Wahl für das Pivot-Element?

- nimmt man das erste, letzte oder mittlere Element kann es bei teilweise vorsortierten Listen schnell zum Worst Case kommen
- der Median bietet sich auch nicht an, weil es aufwendig ist, ihn zu bestimmen
- am einfachsten ist es, ein zufälliges Element zu wählen, da die Chance sehr gering ist, dass es dadurch zum Worst Case kommt

# Aufgabe 3

Sortiere das Array  $A = [42, 17, 12, 15, 4, 11, 31, 14]$  und vergleiche die angewendeten Sortiervverfahren und die Anzahl der Vergleiche.

### 3 a)

Sortiere das Array mit QuickSort und benutze zuerst das letzte Element als Pivot-Element und dann den Median aus den Elementen an der Position  $1, \frac{n}{2}$  und  $n$ .

(Das Pivot-Element wird hier mit Klammern gekennzeichnet.)

1. Pivot = letztes Element

$A = [42, 17, 12, 15, 4, 11, 31, 14]$

- 12, 4, 11, (14), 17, 42, 31, 15 (+7)
- 4, (11), 12, (14), 17, 42, 31, 15 (+2)
- 4, (11), 12, (14), (15), 42, 31, 17 (+3)
- 4, (11), 12, (14), (15), (17), 31, 42 (+2)
- 4, (11), 12, (14), (15), (17), 31, (42) (+1)



## 2. Pivot = Median

$A = [42, 17, 12, 15, 4, 11, 31, 14]$

- 12, 14, 4, 11, (15), 17, 31, 42 (+7 + 3 wegen Median)
- 11, 4, (12), 14, (15), 17, 31, 42 (+3 + 3)
- 4, (11), (12), 14, (15), 17, 31, 42 (+2)
- 4, (11), (12), 14, (15), (17), 31, 42 (+2)
- 4, (11), (12), 14, (15), (17), (31), 42 (+1)

## 3 b)

Sortiere das Array  $A = [42, 17, 12, 15, 4, 11, 31, 14]$  mit den folgenden Algorithmen:

- SelectionSort
- InsertionSort

$A = [42, 17, 12, 15, 4, 11, 31, 14]$

SelectionSort:

- 4, 17, 12, 15, 42, 11, 31, 14 (+7)
- 4, 11, 12, 15, 42, 17, 31, 14 (+6)
- 4, 11, 12, 15, 42, 17, 31, 14 (+5)
- 4, 11, 12, 14, 42, 17, 31, 15 (+4)
- 4, 11, 12, 14, 15, 17, 31, 42 (+3)
- 4, 11, 12, 14, 15, 17, 31, 42 (+2)
- 4, 11, 12, 14, 15, 17, 31, 42 (+1)

$A = [42, 17, 12, 15, 4, 11, 31, 14]$

InsertionSort:

- 17, 42, 12, 15, 4, 11, 31, 14 (+1)
- 12, 17, 42, 15, 4, 11, 31, 14 (+2)
- 12, 15, 17, 42, 4, 11, 31, 14 (+3)
- 4, 12, 15, 17, 42, 11, 31, 14 (+4)
- 4, 11, 12, 15, 17, 42, 31, 14 (+5)
- 4, 11, 12, 15, 17, 31, 42, 14 (+2)
- 4, 11, 12, 14, 15, 17, 31, 42 (+5)

## Vergleiche:

- QuickSort:
  - Pivot = letztes Element: 13
  - Pivot = Median:  $15+6 = 21$
- SelectionSort: 28
- InsertionSort: 21